

PEN-240840116006

Practical-2 Perform Data inspection, cleaning,banding missing values,dupliate removal and data transforation using pandas.

Step: 1 Import Pandas

```
# Import Pandas library

import pandas as pd
```

Step: 2 Create Dataframe

```
# Creating a dictionary containing student data

data = {
    # Student ID
    "Id": [101, 102, 103, 104, 104, 105, 106, 107],

    # Student names
    "Name": ["Rahul", "Priya", "Aakash", None, "Riya", "Kavya", "Himay", "Darshana"],

    # Student ages
    "Age": [20, 21, None, 20, 20, 22, 23, 28],

    # Student cities
    "City": ["Surat", "Navsari", "Surat", "Bharuch", "Bharuch", "Navsari", "Surat", "Ahemedabad"],

    # Student marks
    "Marks": [85, None, 78, 88, 88, 81, 90, 93]
}
```

```
# Create and display a DataFrame
```

```
df = pd.DataFrame(data)
```

```
df
```

	Id	Name	Age	City	Marks	
0	101	Rahul	20.0	Surat	85.0	
1	102	Priya	21.0	Navsari	NaN	
2	103	Aakash	NaN	Surat	78.0	
3	104	None	20.0	Bharuch	88.0	
4	104	Riya	20.0	Bharuch	88.0	
5	105	Kavya	22.0	Navsari	81.0	
6	106	Himay	23.0	Surat	90.0	
7	107	Darshana	28.0	Ahemedabad	93.0	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step: 3 Inspect the Database

```
# Task 1: Display first 5 row of dataframe.
df.head()
```

	Id	Name	Age	City	Marks	
0	101	Rahul	20.0	Surat	85.0	
1	102	Priya	21.0	Navsari	NaN	
2	103	Aakash	NaN	Surat	78.0	
3	104	None	20.0	Bharuch	88.0	
4	104	Riya	20.0	Bharuch	88.0	

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
# Task 2: Display last 5 row of dataframe
df.tail()
```

	Id	Name	Age	City	Marks
3	104	None	20.0	Bharuch	88.0
4	104	Riya	20.0	Bharuch	88.0
5	105	Kavya	22.0	Navsari	81.0
6	106	Himay	23.0	Surat	90.0
7	107	Darshana	28.0	Ahemedabad	93.0

```
# Task 3: Display detail info of dataframe
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  ------  -
0    Id      8 non-null         int64
1   Name     7 non-null         object
2    Age     7 non-null         float64
3    City    8 non-null         object
4   Marks   7 non-null         float64
dtypes: float64(2), int64(1), object(2)
memory usage: 452.0+ bytes
```

```
# Task 4: Generate statistical info.
df.describe()
```

	Id	Age	Marks
count	8.00	7.000000	7.000000
mean	104.00	22.000000	86.142857
std	2.00	2.886751	5.209881
min	101.00	20.000000	78.000000
25%	102.75	20.000000	83.000000
50%	104.00	21.000000	88.000000
75%	105.25	22.500000	89.000000
max	107.00	28.000000	93.000000

```
# Task 5: Display dimension of dataframe.
df.shape
```

```
(8, 5)
```

```
# Task 6: Display name of column.
df.columns
```

```
Index(['Id', 'Name', 'Age', 'City', 'Marks'], dtype='object')
```

```
# Task 7: Display datatype of each column.
df.dtypes
```

```
0
Id      int64
Name    object
Age     float64
City    object
Marks   float64

dtype: object
```

Step 4 : Check missing Values.

```
# Task 1: Check Missing (null) values in every column.
df.isnull().sum()
```

```
0
Id    0
Name  1
Age   1
City  0
Marks 1
```

dtype: int64

Step 5: Fill Missing Values.

```
# Task 1: Fill missing values (replace missing value in marks)
```

```
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

```
# Display the Updated Dataframe.
```

```
df
```

	Id	Name	Age	City	Marks	
0	101	Rahul	20.0	Surat	85.000000	
1	102	Priya	21.0	Navsari	86.142857	
2	103	Aakash	NaN	Surat	78.000000	
3	104	None	20.0	Bharuch	88.000000	
4	104	Riya	20.0	Bharuch	88.000000	
5	105	Kavya	22.0	Navsari	81.000000	
6	106	Himay	23.0	Surat	90.000000	
7	107	Darshana	28.0	Ahemedabad	93.000000	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
# Task 2: Fill missing Age.
```

```
# Replace missing values in the Age column.
```

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

```
# Display the Updated Dataframe.
```

```
df
```

	Id	Name	Age	City	Marks	
0	101	Rahul	20.0	Surat	85.000000	
1	102	Priya	21.0	Navsari	86.142857	
2	103	Aakash	22.0	Surat	78.000000	
3	104	None	20.0	Bharuch	88.000000	
4	104	Riya	20.0	Bharuch	88.000000	
5	105	Kavya	22.0	Navsari	81.000000	
6	106	Himay	23.0	Surat	90.000000	
7	107	Darshana	28.0	Ahemedabad	93.000000	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
# Task 3: Fill missing Name.
```

```
# Replace missing name with the text "Unknown"
```

```
df["Name"] = df["Name"].fillna("Unknown")
```

```
# Display the updated Dataframe.
```

```
df
```

	Id	Name	Age	City	Marks	
0	101	Rahul	20.0	Surat	85.000000	
1	102	Priya	21.0	Navsari	86.142857	
2	103	Aakash	22.0	Surat	78.000000	
3	104	Unknown	20.0	Bharuch	88.000000	
4	104	Riya	20.0	Bharuch	88.000000	
5	105	Kavya	22.0	Navsari	81.000000	
6	106	Himay	23.0	Surat	90.000000	
7	107	Darshana	28.0	Ahemedabad	93.000000	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

Step 6: Detect Duplicate Records.

Task 1: check wheater each row is duplicated.

```
df.duplicated()
```

```

0
0 False
1 False
2 False
3 False
4 False
5 False
6 False
7 False

```

dtype: bool

Task 2: Count total no of duplicate record.

```
df.duplicated().sum()
```

```
np.int64(0)
```

Step 7: Remove Duplicate Records.

Remove duplicate rows from the dataframe.

```
df = df.drop_duplicates()
```

Display the updated Dataframe.

```
df
```

	Id	Name	Age	City	Marks	
0	101	Rahul	20.0	Surat	85.000000	
1	102	Priya	21.0	Navsari	86.142857	
2	103	Aakash	22.0	Surat	78.000000	
3	104	Unknown	20.0	Bharuch	88.000000	
4	104	Riya	20.0	Bharuch	88.000000	
5	105	Kavya	22.0	Navsari	81.000000	
6	106	Himay	23.0	Surat	90.000000	
7	107	Darshana	28.0	Ahemedabad	93.000000	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

Step 8: Rename a column.

```
# Rename the column marks to "score".
# Apply the change directly to the original DataFrame using inplace=True.

df.rename(columns={"Marks": "Score"}, inplace=True)

# Display the updated Dataframe.
df
```

	Id	Name	Age	City	Score	
0	101	Rahul	20.0	Surat	85.000000	
1	102	Priya	21.0	Navsari	86.142857	
2	103	Aakash	22.0	Surat	78.000000	
3	104	Unknown	20.0	Bharuch	88.000000	
4	104	Riya	20.0	Bharuch	88.000000	
5	105	Kavya	22.0	Navsari	81.000000	
6	106	Himay	23.0	Surat	90.000000	
7	107	Darshana	28.0	Ahmedabad	93.000000	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 9: Change Data type.

```
# Change the datatype.
#convert age column float -> integer.

df["Age"] = df["Age"].astype(int)
df.dtypes
```

```
0
Id      int64
Name    object
Age      int64
City    object
Score   float64

dtype: object
```

Step 10: Create a new column.

```
#Create a new column named "final score".

df["Final Score"] = df["Score"]+5

# Display the updated Dataframe.
df
```

	Id	Name	Age	City	Score	Final Score	
0	101	Rahul	20	Surat	85.000000	90.000000	
1	102	Priya	21	Navsari	86.142857	91.142857	
2	103	Aakash	22	Surat	78.000000	83.000000	
3	104	Unknown	20	Bharuch	88.000000	93.000000	
4	104	Riya	20	Bharuch	88.000000	93.000000	
5	105	Kavya	22	Navsari	81.000000	86.000000	
6	106	Himay	23	Surat	90.000000	95.000000	
7	107	Darshana	28	Ahmedabad	93.000000	98.000000	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 11: Convert text to uppercase.

```
# Convert all student name to uppercase.
```

```
df["Name"] = df["Name"].str.upper()
```

```
# Display the updated Dataframe.  
df
```

	Id	Name	Age	City	Score	Final Score	
0	101	RAHUL	20	Surat	85.000000	90.000000	
1	102	PRIYA	21	Navsari	86.142857	91.142857	
2	103	AAKASH	22	Surat	78.000000	83.000000	
3	104	UNKNOWN	20	Bharuch	88.000000	93.000000	
4	104	RIYA	20	Bharuch	88.000000	93.000000	
5	105	KAVYA	22	Navsari	81.000000	86.000000	
6	106	HIMAY	23	Surat	90.000000	95.000000	
7	107	DARSHANA	28	Ahemedabad	93.000000	98.000000	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 12: Convert text to lowercase.

```
#Convert city names to lowercase.  
df["City"] = df["City"].str.lower()
```

```
# Display the updated Dataframe.  
df
```

	Id	Name	Age	City	Score	Final Score	
0	101	RAHUL	20	surat	85.000000	90.000000	
1	102	PRIYA	21	navsari	86.142857	91.142857	
2	103	AAKASH	22	surat	78.000000	83.000000	
3	104	UNKNOWN	20	bharuch	88.000000	93.000000	
4	104	RIYA	20	bharuch	88.000000	93.000000	
5	105	KAVYA	22	navsari	81.000000	86.000000	
6	106	HIMAY	23	surat	90.000000	95.000000	
7	107	DARSHANA	28	ahemedabad	93.000000	98.000000	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 13: Remove Extra spaces.

```
#Remove Extra (white)leading and trailing space from city name.
```

```
df["City"] = df["City"].str.strip()
```

```
# Display the updated Dataframe.  
df
```

	Id	Name	Age	City	Score	Final Score	
0	101	RAHUL	20	surat	85.000000	90.000000	
1	102	PRIYA	21	navsari	86.142857	91.142857	
2	103	AAKASH	22	surat	78.000000	83.000000	
3	104	UNKNOWN	20	bharuch	88.000000	93.000000	
4	104	RIYA	20	bharuch	88.000000	93.000000	
5	105	KAVYA	22	navsari	81.000000	86.000000	
6	106	HIMAY	23	surat	90.000000	95.000000	
7	107	DARSHANA	28	ahemedabad	93.000000	98.000000	

Next steps: [Generate code with df](#) [New interactive sheet](#)

Step 14: Sort the Dataset.

```
# Sort data based on final score.
df_sorted = df.sort_values(by="Final Score",ascending=False)
df_sorted
```

		Id	Name	Age	City	Score	Final Score	
7	107		DARSHANA	28	ahemedabad	93.000000	98.000000	 
6	106		HIMAY	23	surat	90.000000	95.000000	
3	104		UNKNOWN	20	bharuch	88.000000	93.000000	
4	104		RIYA	20	bharuch	88.000000	93.000000	
1	102		PRIYA	21	navsari	86.142857	91.142857	
0	101		RAHUL	20	surat	85.000000	90.000000	
5	105		KAVYA	22	navsari	81.000000	86.000000	
2	103		AAKASH	22	surat	78.000000	83.000000	

Next steps: [Generate code with df_sorted](#) [New interactive sheet](#)

Step 15: Save the cleaned Dataset.

```
#Save clean dataframe as new CSV file.

df.to_csv("Cleaned_Students.CSV",index=False)
print("Dataset created successfully")
```

Dataset created successfully